Operating System Security

Week 4: Windows Security Analysis

How Secure is Windows?

Secure Operating Systems (Revisited)

A **secure operating system's** access enforcement satisfies the reference monitor concept.

The ***reference monitor concept*** defines the properties of any system that securely enforces a **mandatory protection system**, with three guarantees :

1. *Complete mediation*: the access enforcement mechanism mediates all security-sensitive operations
2. *Tamperproof*: the access enforcement mechanism & its protection system, can't be modified by bad processes
3. *Verifiable*: the access enforcement mechanism, including its protection system, must be completely assured (meaning it must be small enough for thorough analysis and testing).

Assessment Criteria of Systems

To assess a system, we must specify how each system enforces the reference monitor guarantees

- Doing this often exposes an insecure operating system, but it is difficult to define how close to "secure" an operating system is.
- We can use a set of questions [*] to evaluate the extent to which an OS satisfies these reference monitor guarantees.
- We use the topics of these questions to guide our discussion.

[*] T. Jaegar, "Operating System Security", Morgan & Claypool Publishers, 2008.

Windows

TCB is extremely large (50 million LOC in the OS alone)

Recent success reducing vulnerabilities through better code development practices, code analysis tools, & more secure configuration settings

Most popular OS = most targeted OS = bad reputation

Is Linux any better?

Windows and Complete Mediation

Complete mediation is referring to how well the access control mechanism is working.

- See the “Windows Access Control” module for a detailed description.

Windows: TCB Tamperproofing and Verification

Verification of TCB:

- Any basis for correctness is informal.
- Windows also has an unbounded TCB, and extensible kernel that prevent any effective formal verification

Tamperproofing

- There are some good and bad things…

Windows TCB

Consists of all system services and processes running as a trusted user (e.g. Administrator)

Provides a setuid-like mechanism

- Also has concern of whether vulnerabilities exist in such services, which would lead to compromise

The complexity of the discretionary Windows access control model often results in users running as Administrator

TCB and Administrative Users

Windows users usually just want their software to install and run properly

- Avoiding access control issues is desired by most users. Solution? Run as administrator (or a user with administrator rights).
- This is bad, as everything becomes part of the TCB.
- User Account Control (UAC) aimed to help with this, and is good as long as users do not disable it
  - Disabling is easy

How is the TCB protected?

Ultimately by having two different modes: user mode and kernel mode.

Typically, it relies on the processor to implement “protection rings”.

- Ensures that privileged operations are only run when the processor is running in a “privileged mode”.

Protection rings

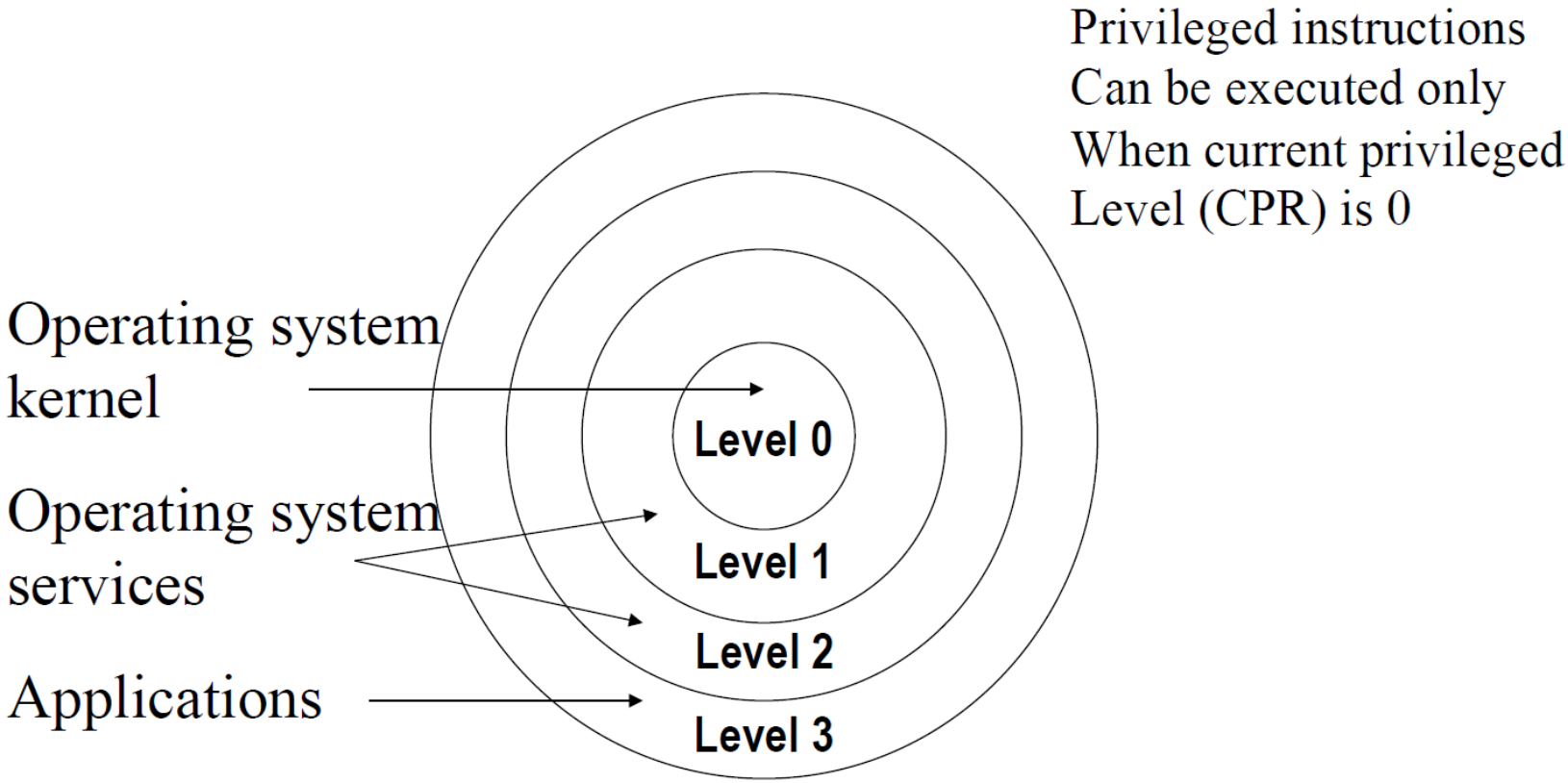
This is a general operating system concept, which applies to most modern OSes

- Same concept of protection rings was first used in UNIX and Multics!

It’s actually implemented by the CPU and made use of by the kernel.

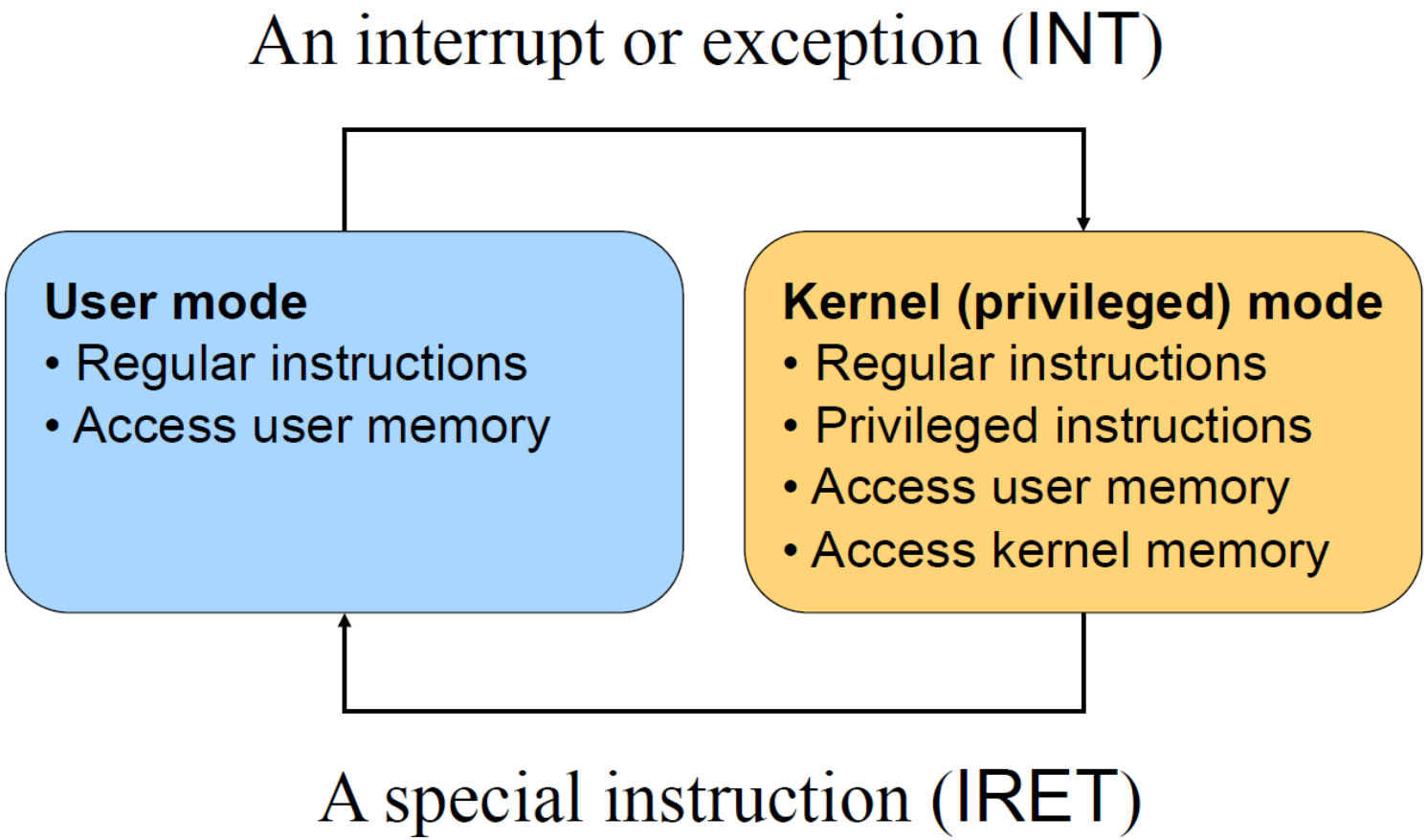
It is how “kernel mode” (aka privileged mode) is enforced.

Protection rings on x86 and x64 CPUs



[*] Courtesy of Andy Bavier, “OS Structure and System Calls”, slides 4-6, 2010, URL: https://www.cs.princeton.edu/courses/archive/fall10/cos318/lectures/OSStructure.pdf , last accessed Jan. 28, 2018.

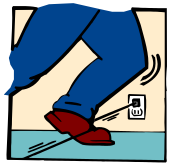
Protection rings: what does it enable?



[*] Courtesy of Andy Bavier, "OS Structure and System Calls", slides 4-6, 2010, URL: https://www.cs.princeton.edu/courses/archive/fall10/cos318/lectures/OSStructure.pdf , last accessed Jan. 28, 2018.

TripWire

A tool for file integrity checking



Based on using cryptographic hashes of files, stores them in a database, and later checks they haven't changed.

Useful for files that "should not change" in e.g., the OS.

Note: this is a tool that provides an additional layer of security, not part of the OS itself.

What are the limitations of this approach?

TCB File Protection: Windows Driver Signing

64-bit versions of Windows Vista (and later) enforce that all Microsoft kernel drivers be digitally signed

- Windows 10: must be signed by an EV certificate issued by a trusted certificate authority, such as Entrust, then submitted to Microsoft for their signature.

The digital signatures of these kernel drivers are then verified by the OS at load-time.

- As a result, drivers that do not have a valid signature will not be loaded into the kernel.
- See the following article for more details: http://msdn.microsoft.com/en-us/library/ff548231%28v=VS.85%29.aspx (last accessed Jan. 23, 2020).

Running TCB Protection: Data Execution Prevention (DEP)

Makes it difficult to exploit a memory vulnerability, by preventing code execution from data pages

Rationale: typically, code is executed from code pages

Marks all memory locations in a process as non-executable (NX) unless the location explicitly contains executable code

Processors that support this can raise an exception when code is executed from a page that is marked as NX

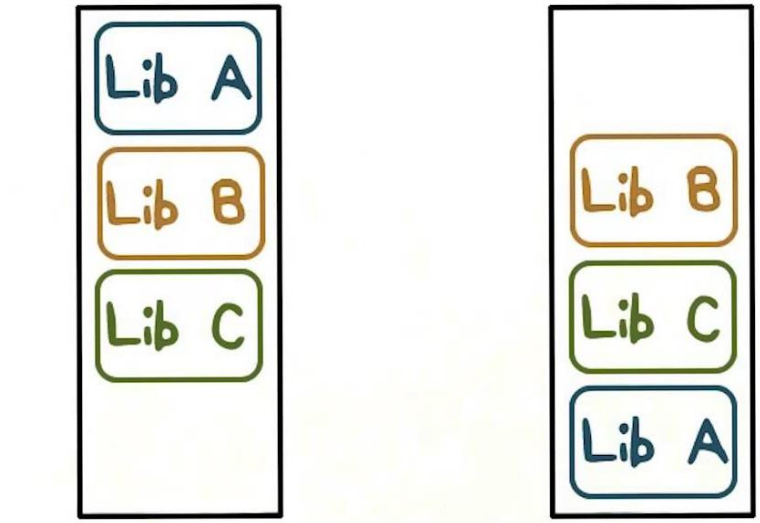
See: “Data Execution Prevention”, https://docs.microsoft.com/en-us/windows/win32/memory/data-execution-prevention, 2018, (last accessed Jan. 23, 2020)

Another Running TCB Protection: “Address Space Layout Randomization” (ASLR)

Makes DEP defeats (e.g., “return oriented programming”) much harder.

Randomizes where processes and libraries are loaded into memory

- Makes exploitation of memory errors much more difficult.



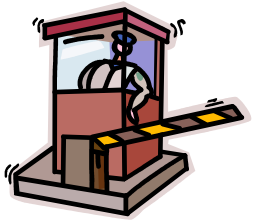
Memory Layout

Windows Security Analysis

Recall we have a system of 7 questions to help understand the security of an operating system

1. Complete mediation: How does the reference monitor interface ensure all security-sensitive operations are mediated *correctly*?
   - Mediation is performed by object managers.
   - Without the source code, it is difficult to know where mediation is performed.

Windows Security Analysis (con't)



2. Complete mediation: Does the reference monitor interface mediate security sensitive operations *on all system resources*?
   - Object managers (modules for new objects created by a program) can be installed into the reference monitor
   - Since the set of managers is extensible, it is feasible that insecure object managers could be added
   - The only way to guarantee complete mediation here is to have a formal approach that defines what each manager does and how it is to be secured.

Windows Security Analysis (con't -2)

3. Complete mediation: How do we *verify* that the reference monitor interface provides complete mediation?
   - No approach has been used.

4. Tamperproof: How does the system protect the *reference monitor, including its protection system*, from modification?
   - Discretionary access control – untrusted user processes can modify permissions
   - Users running as administrator – although UAC helps
   - Kernel modules could be loaded, but later versions of Windows 64-bit OS's enforce only signed drivers

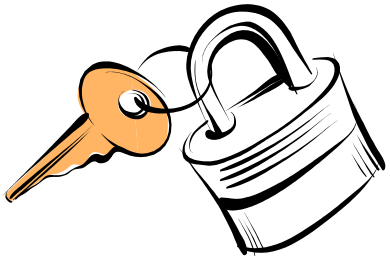
Windows Security Analysis (con't -3)

5. Tamperproof: Does the system's protection system *protect the TCB programs*?
   - No better than UNIX in terms of processes running as root
   - Good (recently done): driver signing, DEP, and ASLR can prevent many attacks

6. Verifiable: What is the basis for the correctness of the *system's TCB*?
   - Any basis for correctness is informal.
   - Windows also has an unbounded TCB, and extensible kernel that prevent any effective formal verification

Windows Security Analysis (con't-4)

7. Verifiable: Does the protection system enforce *the system's security goals*?
   - Discretionary system, so not possible to verify.
   - Since the model is more complex than the Linux/UNIX model, and can be extended arbitrarily, verifying its security is even more difficult.



Summary Questions

Given the security analysis of Windows, how secure do you think it is?

Which of the security analysis points do you think Windows could improve upon (and how)?